

Plan: Corrección SEO del widget Opynio

Documento técnico — análisis del problema, alternativas evaluadas, plan de implementación priorizado.

Proyecto:	Opynio (web.opynio.com) — widget embebido en sitios cliente
Branch propuesta:	fix/widget-seo-facade-2026-05-06
Fecha:	2026-05-06
Síntoma reportado:	Widget de reseñas idéntico en 115 páginas de cliente WordPress está degradando su SEO (~3.000 palabras duplicadas por página, ~60-70% del peso textual).
Cliente afectado (pendiente confirmar):	ICTess (115 páginas de cursos/diplomados).
Severidad:	ALTA — afecta indexación de cliente premium y, por contagio, percepción del producto Opynio.

CONTENIDO

1. Resumen ejecutivo
2. El problema — qué reporta el cliente
3. Análisis técnico — por qué pasa con un widget que ya es JS
4. Alternativas evaluadas y descartadas
5. Solución recomendada — patrón fachada + detección de bot
6. Plan de implementación por fases
7. Cambios concretos en `widget.js`
8. Criterios de éxito y verificación
9. Rollback y mitigaciones
10. Riesgos y supuestos abiertos

1. Resumen ejecutivo

Un cliente de Opynio que embebe el widget de reseñas en 115 páginas de cursos sufre degradación SEO porque Googlebot indexa **el mismo bloque de ~3.000 palabras de reseñas en todas las páginas**. Adicionalmente hay un bug que duplica el bloque dentro de la misma página (las 18 reseñas aparecen 36 veces). Ese bloque pesa proporcionalmente más que el contenido único de cada curso, y además contiene términos no relacionados con el tema de la página (reseñas que mencionan "BMS", "SCADA", "VRV" en una página de Diplomado en Ciencia de Datos), lo que Google lee como incoherencia tópica.

El widget actual *ya carga vía JavaScript asíncrono*, por lo que la solución típica de "lazy load con JS" no aplica: **Googlebot Mobile renderiza con headless Chromium e indexa el HTML post-JS**. La solución real requiere que Google *nunca llegue a ver* el bloque de reseñas, lo cual exige uno de estos enfoques: (a) `iframe` en origen distinto, (b) detección de User-Agent en el loader, o (c) **patrón fachada** (mostrar resumen mínimo, cargar reseñas solo tras interacción del usuario).

Se recomienda implementar **patrón fachada + detección de bot en cascada**, con JSON-LD

`AggregateRating` para preservar el rich snippet en SERPs. Coste estimado: 1-2 días de desarrollo + 1 día de validación. Migración invisible para clientes existentes (mismo snippet, comportamiento mejorado).

2. El problema — qué reporta el cliente

Síntomas reportados

- Bloque de reseñas **idéntico al 100%** en 115 páginas distintas del cliente.
- **Bug de duplicado intra-página**: las 18 reseñas aparecen dos veces seguidas en cada página (36 reseñas en el HTML).
- El bloque representa **el 60-70% del peso textual** de cada página.
- Las reseñas mencionan temas (`BMS` , `SCADA` , `VRV` , `ACS` centralizada) que no coinciden con el título de la página (p. ej. "Diplomado en Ciencia de Datos") → Google detecta incoherencia tópica.
- Resultado: Google está degradando el ranking de las páginas de cursos del cliente.

El embed que el cliente está usando (versión actual de `widget.js`):

```
<div class="opynio-widget"
  data-business-id="..."
  data-type="horizontal-carousel"></div>
<script src="https://web.opynio.com/widget.js" async></script>
```

3. Análisis técnico — por qué pasa con un widget que ya es JS

Premisa importante a corregir

Existe la creencia popular de que "Google no indexa contenido cargado por JS asíncrono en páginas con señales débiles". **Esto era cierto en 2018 — no en 2026.** Googlebot Mobile renderiza con headless Chromium en la primera pasada y espera al evento `load` + un timeout adicional para AJAX. El widget actual ya carga `async` y aun así el cliente está sufriendo el problema. Por tanto, mover el código de `async` a `addEventListener('load')` **no cambia nada**.

El comportamiento actual de `widget.js` (verificado en `public/widget.js`):

1. HTML estático contiene `<div class="opynio-widget" data-business-id="...">` vacío.
2. `widget.js` se carga con `async` , ejecuta `initAll()` en `DOMContentLoaded` .
3. `initWidget` hace `fetch` al endpoint `widget-proxy` de Supabase.
4. Inyecta el HTML completo de las reseñas en el `div` .
5. Googlebot, durante el render, espera al `networkidle` y captura el HTML inyectado → **indexa las reseñas**.

El problema raíz no es *cuándo* se carga el contenido, sino *que se cargue* en una forma que Google pueda capturar.

3.1 Lo que cuenta como "duplicado" para Google

Google evalúa duplicado por bloque de texto, no por página completa. Un bloque de ~3.000 palabras idéntico en N páginas:

- Reduce el "peso" relativo del contenido único de cada página.
- Introduce términos tópicos que contradicen el title/H1 (incoherencia semántica).
- Hace que Google considere las páginas como "thin content" en lo que se refiere a su contenido único.
- Puede activar *Soft 404* o degradación masiva si la proporción de duplicado es muy alta.

3.2 Lo que NO cuenta como duplicado

- Texto corto y específico (rating numérico, contador, marca, CTA).
- Datos estructurados JSON-LD (Google los lee como metadato, no como cuerpo de la página).
- Contenido detrás de interacción explícita (click), si Googlebot no lo desencadena.
- Contenido en iframe de origen distinto (Google lo atribuye al origen del iframe).

4. Alternativas evaluadas y descartadas

Alternativa	Funciona	Por qué (no) funciona
Mover carga a <code>window.load</code>	NO	Googlebot espera al <code>load</code> . El widget ya es async; cambiar el evento no aísla nada.
IntersectionObserver al hacer scroll	NO	Googlebot scrollea durante el render para activar lazy-load de imágenes y contenido. Verá las reseñas igual.
Atributo <code>data-nosnippet</code>	PARCIAL	Solo evita que el bloque aparezca como snippet en SERPs. Google sigue indexando y evaluando duplicidad. Útil como complemento, no como solución.
<code>setTimeout</code> largo (5-10s)	NO	Googlebot tiene timeouts variables pero amplios; espera AJAX en curso. Frágil y degrada UX para usuarios reales.
Shadow DOM (closed)	PARCIAL	Google ha mejorado la indexación de Shadow DOM open. Closed no es estándar y rompe accesibilidad. No fiable.
Bloquear via <code>robots.txt</code> en Opynio	NO	El contenido se inyecta en la página del cliente; el <code>robots.txt</code> de Opynio no afecta lo que ve el bot en el dominio del cliente.
iframe en subdominio (<code>reviews.opynio.com</code>)	SÍ	Aísla el contenido por origen. Garantía técnica completa. Coste alto: nueva infra + cambio de snippet en cliente + auto-resize via <code>postMessage</code> .
Detección de User-Agent en el loader	SÍ	No inyecta HTML si UA es Googlebot/Bingbot/etc. Riesgo bajo de cloaking porque mostramos <i>menos</i> al bot, no contenido distinto.
Patrón fachada (recomendada)	SÍ	Render inicial con resumen mínimo + botón "Ver reseñas". Google no hace click → nunca ve las reseñas. Cero riesgo de cloaking (todos ven la misma fachada inicialmente).

5. Solución recomendada — patrón fachada + detección de bot

Decisión técnica

Implementar **patrón fachada como comportamiento por defecto** de `widget.js`, con **detección de bot** como salvaguarda en cascada y **JSON-LD AggregateRating** para preservar el rich snippet en SERPs sin penalización por duplicado.

5.1 Cómo se ve la fachada

HTML que el bot indexa (mínimo, no duplicado entre páginas porque el rating y contador varían por empresa):

```

<div class="opynio-widget" data-business-id="...">
  <div class="opynio-facade">
    <div class="opynio-facade-rating">4.8 ★</div>
    <div class="opynio-facade-count">132 reseñas verificadas</div>
    <button class="opynio-facade-btn" type="button">Ver reseñas</button>
  </div>
  <script type="application/ld+json">
    {
      "@context": "https://schema.org",
      "@type": "AggregateRating",
      "ratingValue": "4.8",
      "reviewCount": "132"
    }
  </script>
</div>

```

HTML que ve el usuario tras click: el widget completo actual (carrusel horizontal con las 18 reseñas).

5.2 Por qué esto sí evita el problema

Aspecto	Widget actual	Con fachada
HTML que Google indexa	~3.000 palabras idénticas en 115 páginas	~10 palabras (rating + count + CTA)
Detectado como contenido duplicado	Sí	No
Reseñas visibles al usuario	Sí, en pageload	Sí, en 1 click
Googlebot las indexa	Sí	No (no hace click)
Riesgo de cloaking	—	Bajo (ambos ven misma fachada inicialmente)
Performance (LCP, INP)	Carga JSON + render en pageload	Cero coste hasta interacción
Rich snippet en SERPs	Sin schema	Sí, vía JSON-LD AggregateRating

5.3 Detección de bot como segunda barrera

Aunque la fachada es suficiente por sí sola, añadimos detección de UA conocidos (Googlebot, Bingbot, AhrefsBot, etc.) en `initWidget`: **si UA es bot, no hacemos ni siquiera la fachada interactiva — solo el JSON-LD y el rating mínimo**. Esto cubre el caso (improbable) de que algún bot futuro decida hacer click en botones.

5.4 Tradeoffs honestos

- **UX cambia para usuarios reales:** las reseñas requieren 1 click. Esperable bajada del 20-40% en visualizaciones del bloque completo, compensada por mejor SEO del cliente y mejor perf.

- **Cliente debe aceptar el cambio:** comunicar valor ("tu SEO se recupera, tu sitio carga más rápido"). Para clientes que prefieran el comportamiento actual, dejamos opt-out con `data-facade="false"`.
- **Rich snippet:** el JSON-LD AggregateRating mantiene el rating visible en SERPs como antes (incluso lo mejora si no estaba presente).

6. Plan de implementación por fases

Fase	Prioridad	Esfuerzo	Entregable
1. Detección de bot (parche express)	P0	~30 min	<code>widget.js</code> con UA-check que evita inyección si bot. Detiene el sangrado en cliente afectado.
2. Patrón fachada por defecto	P1	~1 día	Fachada con rating + count + botón. JSON-LD inyectado. Click monta el widget completo. Flag <code>data-facade="false"</code> para opt-out.
3. Bug del duplicado intra-página	P1	Pendiente reproducir	Investigar URL real del cliente. Verificar si el plugin/shortcode WordPress emite el div doble, o si <code>MutationObserver</code> en <code>widget.js</code> dispara doble <code>initAll</code> .
4. Validación con cliente afectado	P1	~2 días tras deploy	Verificar via Search Console que páginas del cliente recuperan ranking. Inspección URL en GSC.
5. Comunicación y opt-in inverso	P2	~0.5 día	Email a clientes activos: "Cambio invisible para SEO; si quieres comportamiento anterior, añade <code>data-facade="false"</code> ."
6. Plan B: iframe en <code>reviews.opynio.com</code>	P3	~1 día	Solo si tras 4 semanas la fachada no resulta suficiente. Endpoint nuevo + <code>postMessage</code> resize + snippet alternativo.

7. Cambios concretos en `widget.js`

7.1 Constantes nuevas (al inicio del IIFE)

```
var BOT_REGEX = /Googlebot|bingbot|AhrefsBot|SemrushBot|DuckDuckBot|Slurp|Baiduspider|YandexBot|fa
var FACADE_DEFAULT = true;
var BOT_DETECTED = BOT_REGEX.test(navigator.userAgent || '');
```

7.2 Nuevo renderer de fachada

```
function renderFacade(el, business, s, onActivate) {
  var rating = (business.avg_rating || 0).toFixed(1);
  var count = business.review_count || 0;
  var stars = generateStars(business.avg_rating);
  el.innerHTML = ''
    + '<div class="opynio-facade">'
    + '  <div class="opynio-facade-summary">'
    +    '<div class="opynio-facade-rating">' + rating + '</div>'
    +    '<div class="opynio-stars">' + stars + '</div>'
    +    '<div class="opynio-facade-count">' + count + ' ' + s.reviews + '</div>'
    +  '</div>'
    +  '<button type="button" class="opynio-facade-btn">' + s.seeMore + '</button>'
    + '</div>'
    + '<script type="application/ld+json">{'
    +   '"@context":"https://schema.org",'
    +   '"@type":"AggregateRating",'
    +   '"ratingValue":"' + rating + ','
    +   '"reviewCount":"' + count + '"'
    + '}</script>';

  var btn = el.querySelector('.opynio-facade-btn');
  if (btn && !BOT_DETECTED) {
    btn.addEventListener('click', function(e) {
      e.preventDefault();
      onActivate();
    });
  }
}
```

7.3 Modificación de `initWidget`

```
async function initWidget(el) {
  if (el.dataset.loaded) return;
  el.dataset.loaded = 'true';

  var businessId = el.dataset.businessId;
  var type = el.dataset.type || 'badge';
  var theme = el.dataset.theme || 'light';
  var facadeEnabled = el.dataset.facade !== 'false'; // ← opt-out

  if (!businessId) { renderError(el, 'data-business-id requerido'); return; }
  el.className = 'opynio-widget opynio-theme-' + theme;
  renderLoader(el);

  try {
    var data = await fetchData(businessId);
    if (!data.business) { renderError(el, 'Negocio no encontrado'); return; }

    var targetLang = el.dataset.lang || detectTargetLang();
    var s = await getStrings(targetLang);

    // Tipos sin fachada (badge/floating/sidebar): comportamiento original
    if (['badge', 'floating', 'sidebar'].indexOf(type) !== -1) {
      renderers[type](el, data.business, [], s);
      return;
    }

    // Para los demás tipos: fachada por defecto
    if (facadeEnabled) {
      renderFacade(el, data.business, s, function () {
        renderFullWidget(el, data, type, theme, targetLang, s);
      });
      return;
    }

    // Opt-out: render directo
    renderFullWidget(el, data, type, theme, targetLang, s);
  } catch (err) {
    renderError(el, err.message);
  }
}
```


7.4 Refactor de `renderFullWidget`

```
async function renderFullWidget(el, data, type, theme, targetLang, s) {
  el.className = 'opynio-widget opynio-theme-' + theme;
  renderLoader(el);
  var reviews = (data.reviews || []).filter(function(r) { return r.review_text; });
  if (reviews.length > 0) reviews = await translateReviews(reviews, targetLang);
  if (reviews.length === 0) {
    el.innerHTML = '<div style="text-align:center;padding:2rem;">'
      + s.noReviewsText + '</div>';
    return;
  }
  if (renderers[type]) {
    renderers[type](el, data.business, reviews, s);
  } else {
    renderError(el, 'Tipo de widget no soportado: ' + type);
  }
}
```

7.5 CSS adicional para la fachada

```
.opynio-facade {
  display: flex; align-items: center; justify-content: space-between;
  padding: 1.25rem 1.5rem; background: var(--card-bg);
  border: 1px solid var(--border-color); border-radius: 12px;
  box-shadow: var(--shadow);
}
.opynio-facade-summary { display: flex; align-items: center; gap: 0.75rem; }
.opynio-facade-rating { font-size: 1.5rem; font-weight: 700; color: var(--text-color); }
.opynio-facade-count { font-size: 0.85rem; color: var(--subtext-color); }
.opynio-facade-btn {
  background: var(--opynio-green); color: white; border: none;
  padding: 0.5rem 1rem; border-radius: 8px; font-weight: 600;
  cursor: pointer; transition: background 0.2s;
}
.opynio-facade-btn:hover { background: var(--opynio-green-dark); }
```

8. Criterios de éxito y verificación

8.1 Inmediato (post-deploy, ~24h)

- En una página del cliente afectado, `curl -A "Googlebot" <url>` devuelve HTML **sin las reseñas completas**, solo con el bloque fachada + JSON-LD.
- En navegador real, el widget muestra fachada y al click monta el carrusel completo igual que antes.
- Performance: LCP de la página del cliente mejora (no hay fetch + render del JSON pesado en pageload).

8.2 A 2-4 semanas

- Search Console: páginas del cliente afectado recuperan posiciones. URL Inspection muestra contenido renderizado sin las reseñas.
- Reportes de "Cobertura": no aparecen como duplicate content / soft 404.
- Métricas internas del widget: tasa de click en fachada (esperado 15-30% según industria).

8.3 Métrica anti-objetivo

Si tras 4 semanas el cliente afectado no recupera ranking, escalar a Plan B (iframe en subdominio). Esto sería señal de que (a) el daño SEO es ya estructural y necesita reconsideración, o (b) Google está renderizando con interacción simulada en algún subset de bots.

9. Rollback y mitigaciones

Rollback inmediato

El cambio está completamente contenido en `widget.js` y `widget.js` se sirve desde nuestro CDN/servidor con caché de baja TTL. Revertir a la versión anterior es **1 commit + invalidar caché** (~5 min). No hay migración de datos, no hay cambios en BBDD, no hay cambios en API.

Mitigaciones intermedias si se detectan problemas:

- **Cliente concreto reporta UX rota:** aplicar `data-facade="false"` en su embed (volver al comportamiento actual sólo para ese cliente).
- **Bot real no detectado:** añadir su UA al `BOT_REGEX` y redeployer (~2 min).
- **El JSON-LD da error de schema:** el widget sigue funcionando; el JSON-LD es independiente y aislado en `<script type="application/ld+json">` .

10. Riesgos y supuestos abiertos

Riesgo / supuesto	Impacto	Mitigación
El cliente embebe el widget vía un plugin de WordPress que emite el div dos veces (bug de duplicado intra-página).	Alto	Reproducir en URL real, identificar fuente. Si es plugin del cliente: educación; si es <code>MutationObserver</code> nuestro: parche en <code>initAll</code> .
Google considera la fachada como "cloaking" si interpreta que el botón oculta contenido relevante.	Medio	El botón es claramente etiquetado, accesible, y la mayoría de embeds populares (YouTube facade, Disqus lazy, Twitter embed) usan este patrón sin penalización. Riesgo es bajo en la práctica.
Tasa de click en fachada baja respecto al render automático actual → caída percibida de "engagement".	Medio	Mostrar rating + estrellas + count visualmente atractivos en la fachada. Botón con CTA clara. Métrica esperada: 15-30%.
Algún cliente activamente quiere las reseñas siempre visibles (caso landing de venta).	Bajo	Opt-out vía <code>data-facade="false"</code> . Documentar en página de configuración del widget.
El bug del duplicado intra-página pueda venir del MutationObserver de <code>widget.js</code> que dispara <code>initAll()</code> en cualquier mutación del body.	Alto	Auditar línea 599-605 de <code>widget.js</code> . Posible parche: comprobar <code>data-loaded</code> antes de re-init y debounce el observer.
Cliente afectado (asumido ICTess) embebe el HTML renderizado y no el script (caso poco probable pero posible).	Alto	Verificar con <code>view-source:</code> en una URL real antes de implementar. Si es así, la solución es educación al cliente, no código.

Anexo A — Decisiones pendientes de confirmar con stakeholder

1. Cliente afectado confirmado → ICTess?
2. Acceso a una URL real del cliente para auditar el HTML renderizado y reproducir el bug del duplicado.
3. ¿Activar fachada por defecto para todos los clientes, o solo para el afectado en una primera ronda?
4. ¿Comunicar el cambio proactivamente a los clientes activos, o solo a los que pregunten?
5. Plan B (iframe): subdominio `reviews.opynio.com` o path `web.opynio.com/embed/{id}` ?